

Servomotores Serie M17 - HOJA DE DATOS



Control de movimiento todo en uno, sencillo y asequible
De la educación a la innovación



Introducción

Los Servomotores Serie M17 son soluciones de control de movimiento todo en uno que integran un motor, un controlador de motor (driver), un controlador de movimiento y un encoder en un único paquete compacto. Estos servomotores cuentan con una interfaz de comunicación RS-485, lo que permite conectar varias unidades en cadena (daisy-chain) y controlarlas desde un único punto de conexión. Disponible en cuatro modelos - M17-60, M17-48, M17-40 y M17-34 -, la serie ofrece opciones flexibles para satisfacer requisitos específicos de par y tamaño, manteniendo características de control uniformes. Cada Servomotor Serie M17 incorpora funciones de control avanzadas, incluidos múltiples modos de operación, capacidad de autocalibración y monitoreo de estado integrado mediante indicadores LED. Los motores pueden controlarse fácilmente desde cualquier plataforma, incluidos Mac, PC, Raspberry Pi o Arduino (solo se requiere un adaptador RS485 de bajo costo), lo que los hace ideales tanto para aplicaciones educativas como industriales. La serie admite control de posición preciso con perfiles de movimiento trapezoidales, modo de control en lazo cerrado y un manejo integral de errores. Con dimensiones de montaje estandarizadas NEMA 17, un amplio rango de tensión (12-24V) y un protocolo de comunicación robusto, los Servomotores Serie M17 ofrecen una solución fiable y flexible para aplicaciones que requieren control de movimiento preciso, desde robótica y máquinas CNC hasta equipos de prueba automatizados e instrumentos científicos.

Envíenos sus comentarios

Si encuentra errores en este documento o tiene preguntas, háganoslo saber en: tutorial.gearotons.com/feedback. Agradeceremos mucho sus sugerencias sobre cómo podemos mejorar el producto o la documentación. ¡Muchas gracias!

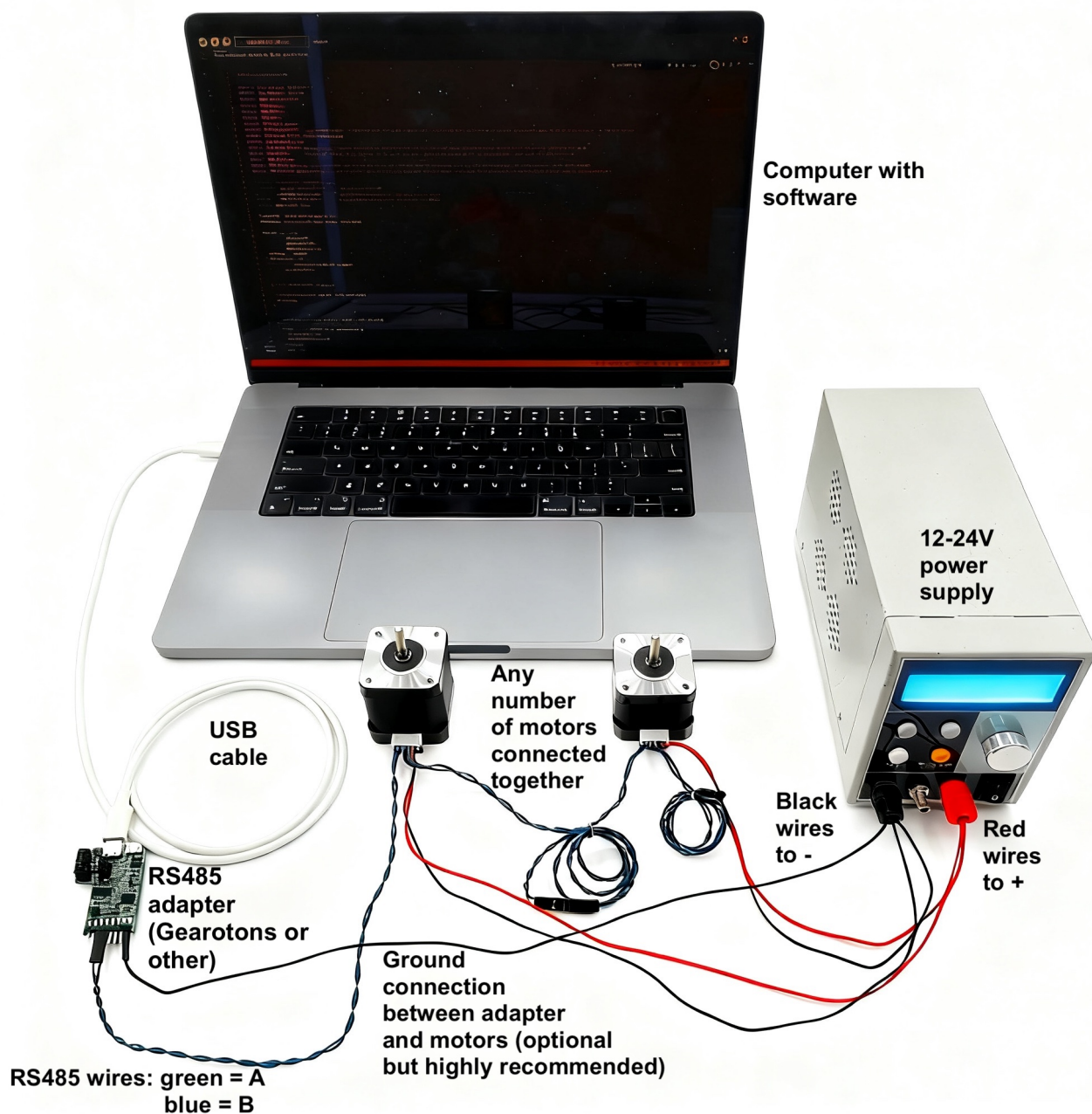


Visite nuestra página de comentarios

Características principales

- Alto nivel de integración que combina un motor, un controlador de motor (driver), un sistema de control de movimiento y un encoder
- Controle cualquier cantidad de motores desde un único controlador sencillo a través de una interfaz de comunicación RS-485
- Control mediante comandos de alto nivel como "Enable mosfets" (habilitar MOSFETs) y "Trapezoid move" (movimiento trapezoidal), evitando el control DIR/STEP con temporización crítica
- Formato compacto de tamaño casi idéntico al de un motor paso a paso NEMA 17 con las mismas especificaciones (sin protuberancias)
- Dimensiones de montaje estandarizadas NEMA 17
- Amplio rango de tensión (12-24V) para opciones de alimentación flexibles
- Control en lazo cerrado de alta precisión con encoder integrado y lazo de control PID que se ejecuta a 32 kHz
- Mucho más eficiente energéticamente en control de lazo cerrado que un motor paso a paso convencional
- Protección integrada contra sobrecorriente, sobretensión y sobretemperatura
- La velocidad máxima puede alcanzar 560 RPM
- La relación par-peso es la misma que la de un motor paso a paso equivalente
- Compatible con una amplia variedad de interfaces y hardware, como Raspberry Pi, Arduino, ESP32, Mac y PC
- Nos esforzamos por ofrecer documentación y tutoriales excelentes para que pueda empezar a trabajar rápidamente
- Ofrecemos documentación compatible con IA por si desea que su IA favorita haga todo el trabajo por usted
- Adecuado para robótica, CNC, automatización, instrumentos científicos, útiles de prueba, impresoras 3D y mucho más
- Disponible en distintos tamaños para que pueda encontrar el par y el precio adecuados para su aplicación

Diagrama de conexión



Sistema de unidades

Los Servomotores Serie M17 utilizan ciertas unidades internas para poder realizar de forma eficiente los cálculos asociados al movimiento (mediante aritmética de enteros). Es responsabilidad del software de control admitir múltiples unidades de medida para las distintas magnitudes. Nuestras bibliotecas de Python y Arduino gestionan las conversiones de unidades automáticamente, lo que le permite trabajar con sus unidades preferidas. A continuación se indican las unidades admitidas para cada magnitud:

Magnitud	Unidades disponibles
Tiempo	timesteps, seconds, milliseconds, minutes, microseconds
Posición	shaft rotations, degrees, radians, encoder counts
Velocidad	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Aceleración	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Corriente	internal current units, milliamps, amps
Tensión	millivolts, volts
Temperatura	celsius, fahrenheit, kelvin

Guía de inicio rápido

Para ayudarle a comenzar con su Servomotor Serie M17, ofrecemos una guía en línea completa que abarca desde la configuración inicial hasta implementaciones avanzadas del protocolo. Esta guía incluye:

- Instrucciones de configuración paso a paso
- Documentación detallada del protocolo de comunicación
- Ejemplos de programación y fragmentos de código
- Descripción de los códigos de error
- Consejos de solución de problemas y buenas prácticas



Haga clic aquí para ver la guía de inicio: tutorial.gearotons.com

LEDs indicadores y botones

El servomotor tiene dos LEDs de estado (verde y rojo). El LED verde parpadea lentamente para indicar el latido (heartbeat) y rápidamente para indicar que se está ejecutando el bootloader en lugar de la aplicación. El LED rojo se enciende brevemente para señalar comunicación en el bus e indica los códigos de error fatal parpadeando un determinado número de veces.

El servomotor tiene dos botones etiquetados "Reset" (reinicio) y "Test" (prueba). El botón Reset reinicia el microcontrolador interno y todo el estado vuelve a los valores predeterminados. El botón Test hace que el motor gire. Presiónelo brevemente para que gire en un sentido, y presiónelo durante más de 0.3 segundos y suéltelo para que gire en el sentido contrario. Manténgalo presionado al menos 2 segundos y suéltelo para que el motor pase al modo de lazo cerrado. Manténgalo presionado durante más de 15 segundos y suéltelo para que el motor realice una calibración sobre sí mismo. Tenga en cuenta que el motor girará durante la calibración y debe poder girar libremente para que esta sea exitosa, por lo que debe retirar cualquier carga antes de realizar esta operación.

Descripción general del protocolo de comunicación

Los Servomotores Serie M17 utilizan RS-485 para la comunicación, lo que permite conectar varios motores en cadena (daisy-chain). Cada motor tiene una dirección única de 64 bits asignada de fábrica, de modo que puede direccionarse individualmente en un bus RS-485. A cada motor también se le puede asignar un alias único de 1 byte (0-251) para ahorrar bytes en el paquete de comunicación manteniendo un control individual similar. Existe además un alias de difusión (broadcast), que es el 255. El protocolo admite un conjunto completo de comandos para control de movimiento, configuración y monitoreo de estado. Se admite la actualización del firmware a través de la interfaz RS-485. La velocidad de comunicación está fijada en 230400 baud.

Resumen de referencia de comandos

Para consultar la fuente de referencia actualizada de todos los comandos disponibles, puede revisar este documento.



https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

También puede ejecutar este comando:

```
pip3 install servomotor # run this just once to install the library and programs
servomotor_command -c
```

Esto imprimirá la información contenida en el archivo motor_commands.json de una forma más legible y proporcionará información de uso para enviar comandos al motor desde la línea de comandos.

Control básico

Comando	Descripción
Disable MOSFETs	Deshabilita los MOSFETs (tenga en cuenta que los MOSFETs están deshabilitados de forma predeterminada tras el encendido inicial).
Enable MOSFETs	Habilita los MOSFETs.
Reset time	Restablece el tiempo absoluto a cero (llame a este comando primero, antes de emitir cualquier comando de movimiento).
Emergency stop	Realiza una parada de emergencia (detiene todo movimiento, deshabilita los MOSFETs y vacía la cola).
Zero position	Establece la posición actual como la posición cero (el origen).
System reset	Reinicia el sistema y entra en el bootloader. El motor se reinicia de inmediato y entra en el bootloader. Si no se envía ningún comando en un breve periodo de tiempo, el motor sale del bootloader y ejecuta la aplicación desde el principio.

Control de movimiento

Comando	Descripción
Trapezoid move	Se mueve inmediatamente a la posición indicada usando la velocidad configurada actualmente (la velocidad se establece con un comando aparte).
Go to position	Se mueve a la posición indicada en el tiempo especificado. Se aplican aceleración y desaceleración para que el movimiento sea suave.
Homing	Realiza el homing del motor (es decir, se mueve hasta chocar y entonces se detiene de inmediato).
Go to closed loop	Pasa al modo de control de posición en lazo cerrado.
Move with acceleration	Hace girar el motor con la aceleración especificada.
Move with velocity	Hace girar el motor con la velocidad especificada.
Multimove	Compone varios movimientos para ejecutarse uno tras otro. Tenga en cuenta que si la cola queda vacía después de ejecutarse todos los movimientos y el motor no está detenido, se produce un error fatal.

Configuración

Comando	Descripción
Set maximum velocity	Establece la velocidad máxima (no se utiliza por el momento).
Set maximum acceleration	Establece la aceleración máxima.
Start calibration	Inicia una calibración, que determina los valores promedio de los sensores Hall y si están funcionando correctamente.
Set maximum motor current	Establece la corriente máxima del motor y la corriente máxima de regeneración. Los valores surten efecto de inmediato, pero no se guardan en la memoria no volátil, por lo que vuelven a los valores predeterminados del firmware tras un reinicio.
Set safety limits	Establece los límites de seguridad (para evitar que el movimiento exceda los límites configurados).
Test mode	Establece o activa un modo de prueba determinado. Por el momento, esto está poco documentado. No lo utilice a menos que sea un desarrollador trabajando en casos de prueba. Modos destacados: 0 = borra todos los modos de prueba; 1-9 = modos de prueba del motor; 10-13 = modos de prueba de los LEDs (bloquean el dispositivo); 14-73 = activan los errores fatales 0-59; 74 = fija el umbral de protección contra sobretensión en 22 V (prueba de producción; debería dispararse con una alimentación de 24 V); 75 = fija el umbral de protección contra sobretensión en 26 V (prueba de producción; no debería dispararse con una alimentación de 24 V); 76 = eleva el corte por sobretensión a unos 90 grados C (prueba de margen térmico de producción; el corte predeterminado es de unos 80 grados C). Un reinicio del sistema restaura los umbrales predeterminados de sobretensión y sobretensión.
Set PID constants	Establece las constantes PID del lazo de control que intenta mantener la trayectoria de movimiento.
Set max allowable position deviation	Establece el número de micropasos que la posición real del motor (medida por los sensores Hall) puede desviarse de la posición deseada. Si se supera este valor, se produce un error fatal.
CRC32 control	Habilita o deshabilita la verificación CRC32 de los comandos.

Estado y monitoreo

Comando	Descripción
Get current time	Obtiene el tiempo absoluto actual.
Get n queued items	Obtiene el número de elementos que hay actualmente en la cola de movimientos (si este valor crece demasiado, no encole más comandos de movimiento).
Get hall sensor position	Obtiene la posición medida por los sensores Hall (esta debería ser la posición real del motor; si todo está bien, será aproximadamente igual a la posición deseada).
Get status	Obtiene el estado del motor.
Control hall sensor statistics	Activa o desactiva la recopilación de estadísticas de los sensores Hall y restablece las estadísticas.
Get hall sensor statistics	Lee las estadísticas recopiladas de los sensores Hall. Esto resulta útil para comprobar el estado de los sensores Hall y el ruido presente en el sistema.
Get position	Obtiene la posición deseada actual (que puede diferir ligeramente de la posición real medida por los sensores Hall).
Get comprehensive position	Obtiene la posición deseada del motor, la posición según los sensores Hall y la posición del encoder externo, todo en una sola lectura.
Get supply voltage	Obtiene la tensión medida de la fuente de alimentación.
Get max PID error	Obtiene los valores mínimo y máximo de error observados en el lazo de control PID desde la última lectura.
Get temperature	Obtiene la temperatura medida del motor.
Get debug values	Obtiene valores de depuración, incluidos parámetros de control del motor, tiempos del generador de perfiles, datos de los sensores Hall y otra información de diagnóstico.
Get communication statistics	Obtiene y, opcionalmente, restablece el contador de errores CRC32.

Gestión del dispositivo

Comando	Descripción
Time sync	Envía el tiempo maestro al motor para que pueda sincronizar su propio reloj (hágalo 10 veces por segundo).
Get product specs	Obtiene la frecuencia de actualización (el recíproco del paso de tiempo).
Detect devices	Detecta todos los dispositivos conectados a la interfaz RS485. Los dispositivos se identifican en un instante aleatorio dentro de un segundo. Una colisión es posible pero poco probable. Puede repetir la operación si sospecha que hubo una colisión (por ejemplo, si tiene dispositivos conectados pero no fueron descubiertos en uno o dos segundos).
Set device alias	Establece el alias del dispositivo.
Get product info	Obtiene la información del producto.
Firmware upgrade	Actualiza la memoria flash del servomotor. Antes de emitir un comando de actualización de firmware, debe realizar algunos cálculos, como se muestra en los ejemplos.
Get product description	Obtiene la descripción del producto.
Get firmware version	Obtiene la versión del firmware o la versión del bootloader, según el modo en que se encuentre el dispositivo. Este comando también devuelve los bits de estado, donde el bit menos significativo indica si el dispositivo está actualmente en el bootloader (=1) o en el firmware principal (=0).
Ping	Envía una carga útil con datos arbitrarios y el dispositivo responde con los mismos datos.
Vibrate	Hace que el motor varíe la tensión rápidamente y, por lo tanto, vibre (o deje de vibrar).
Identify	Identifique su motor enviando este comando. El LED verde del motor parpadea rápidamente durante 3 segundos.

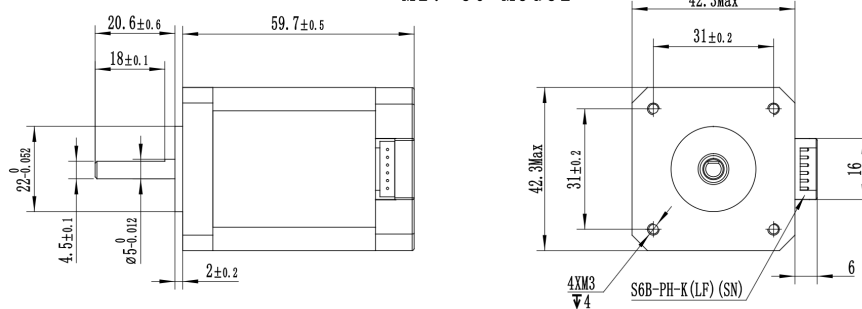
Otros

Comando	Descripción
Capture hall sensor data	Comienza a enviar datos de los sensores Hall (trabajo en curso; no envíe este comando).
Read multipurpose buffer	Lee el contenido del búfer multipropósito (el búfer se utiliza para los datos generados durante la calibración, al pasar al modo de lazo cerrado y al capturar datos de los sensores Hall).

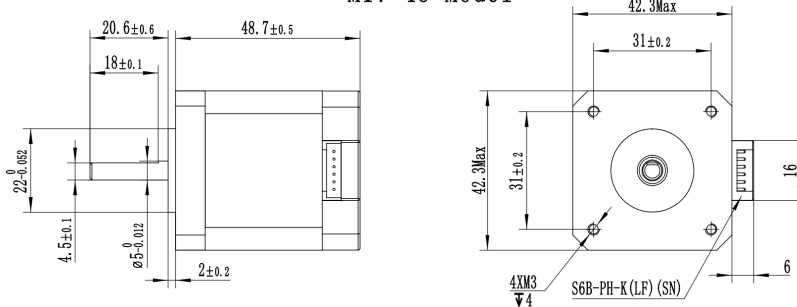
Especificaciones mecánicas

Parámetro	M17-60	M17-48	M17-40	M17-34
Dimensiones (LxA)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Altura	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Longitud del eje	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Peso	470g	360g	285g	210g
Clase de protección	IP20	IP20	IP20	IP20

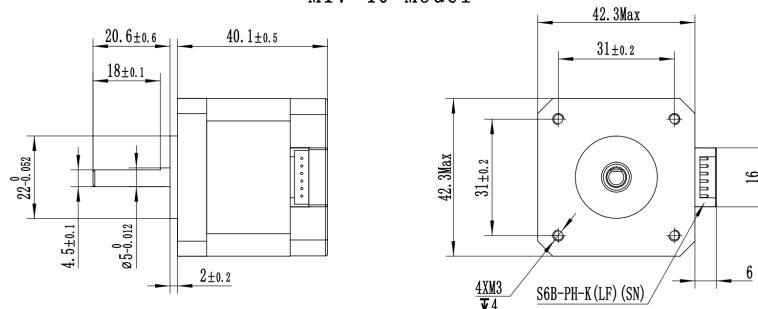
M17-60 Model



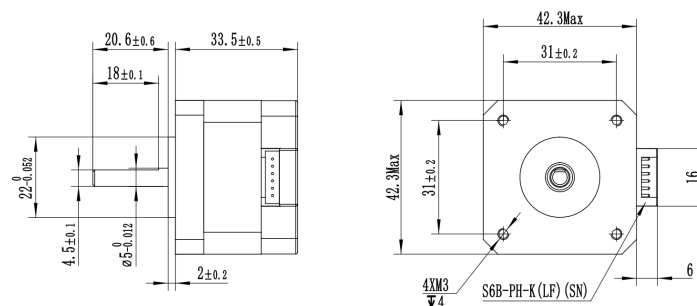
M17-48 Model



M17-40 Model



M17-34 Model



Especificaciones técnicas

Parámetro	M17-60	M17-48	M17-40	M17-34
Tensión de operación	12-24V	12-24V	12-24V	12-24V
Par nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Velocidad máxima	560 RPM	560 RPM	560 RPM	560 RPM
Corriente máxima	1.1A	1.1A	1.1A	1.0A
Potencia nominal	26.4W	26.4W	26.4W	24.0W

Condiciones de operación

Parámetro	Especificación
Temperatura de operación	0C a +80C
Temperatura de almacenamiento	-20C a +60C
Rango de humedad	20% a 80% RH (sin condensación)
Entorno de instalación	Solo para uso en interiores

Biblioteca de Python

Nuestra biblioteca de Python ofrece una interfaz completa para controlar los Servomotores Serie M17. La biblioteca gestiona todos los protocolos de comunicación de bajo nivel y las conversiones de unidades, lo que facilita integrar el control de servomotores en sus proyectos de Python.

Instalación

Instale la biblioteca servomotor usando pip:

```
pip3 install servomotor
```

Ejemplo: control básico del motor

El siguiente ejemplo muestra cómo conectarse a un servomotor, habilitar los MOSFETs y realizar un movimiento trapezoidal:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X' # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110" # Serial device path; change if needed (e.g., "COM3" on
# Windows)
DISPLACEMENT_ROTATIONS = 1.0 # 1 rotation
DURATION_SECONDS = 1.0 # 1 second
DELAY_MARGIN = 0.10 # +10% wait margin because the motor's clock is not
# perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
# should prompt you for the serial port or it will use
# the last used port from a file

servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Características principales

- Compatibilidad con todos los comandos del motor
- Conversión automática de unidades de tiempo, posición, velocidad, aceleración, corriente, tensión y temperatura
- Manejo de errores y monitoreo de estado
- Compatibilidad con varios motores en el mismo bus
- Comience a trabajar con solo unas pocas líneas de código

Documentación

Para obtener la documentación completa de la API y más ejemplos, visite:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Biblioteca de Arduino

La biblioteca de Arduino para los Servomotores Serie M17 ofrece una interfaz fácil de usar para controlar los motores desde placas Arduino. La biblioteca admite comunicación serie por hardware e incluye todas las funciones esenciales de control del motor.

Instalación

Instale la biblioteca mediante el Gestor de Bibliotecas de Arduino:

- 1. Abra el IDE de Arduino
- 2. Vaya a Programa → Incluir Librería → Administrar Bibliotecas
- 3. Busque "Servomotor"
- 4. Localice la biblioteca llamada "Servomotor by Gearotons" y haga clic en Instalar



Instalación manual

Como alternativa, si es un usuario experto, puede obtener el código de la biblioteca de Arduino aquí: https://github.com/tomrodinger/Servomotor_Arduino_Library

Ejemplo: control básico del motor

Este ejemplo muestra cómo inicializar un motor, habilitar los MOSFETs y realizar un movimiento trapezoidal:

```

// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
// enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X' // Device alias
#define BAUD 230400 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f // 1 second
#define TOLERANCE_PERCENT 10 // +10% wait margin because the motor's clock is not
// perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4 // TX pin to RS485 transceiver
#define RS485_RXD 5 // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}

```

Nuestra misión

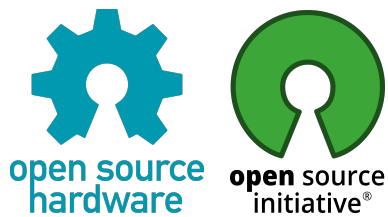
En Gearotons creemos que el futuro pertenece a quienes comprenden la inteligencia artificial y los sistemas de automatización y pueden trabajar junto a ellos. Nuestra misión es empoderar a la próxima generación de innovadores haciendo que la tecnología avanzada de control de movimiento sea accesible, comprensible y asequible para makers, educadores e ingenieros de todo el mundo. Estamos comprometidos con derribar las barreras que tradicionalmente han separado a estudiantes y aficionados de las herramientas de automatización de nivel profesional. Al ofrecer soluciones integradas que eliminan la complejidad sin sacrificar capacidad, hacemos posibles experiencias de aprendizaje prácticas que preparan a las mentes jóvenes para un mundo donde la creatividad humana y la precisión de las máquinas trabajan en armonía. Nuestra convicción fundamental es que todo estudiante debería tener la oportunidad de experimentar con los componentes básicos de la automatización moderna, desde la robótica y los sistemas CNC hasta los instrumentos científicos. A través de nuestras alianzas educativas y nuestro enfoque de código abierto, estamos formando una generación que no solo usa la tecnología, sino que realmente la comprende y le da forma. Juntos estamos construyendo los cimientos para los innovadores que definirán un futuro asombroso.

Código abierto

Creemos en mejorar el mundo a través de la tecnología. El software, el firmware y los archivos de diseño de PCB están disponibles aquí:



<https://github.com/tomrodinger/servomotor>



Versión de la hoja de datos: 1.9 Fecha de publicación: Jun. 29, 2026

(c) 2026 Todas las especificaciones están sujetas a cambios sin previo aviso.